RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

Report on the first practical assignment

Study course "Fundamentals of artificial intelligence"

Team number: Team 26
Students:
Ngoc Bao Tram, Tran, 231ADB294
Kamalesh Ashokkumar Kowsalya 221ADB216
Erwan Matthieu Antoine, Chardey, 250AEM002
Bakhodir, Rasulov, 231ADB155
Bahram Mansurzade 231ADB159

Teaching staff: Alla Anohina-Naumeca

Project/code link: LINK

https://github.com/Tramdanglamduoc/GAMETREE_TEAM9

https://drive.google.com/drive/folders/1-dhYSw35UgR4uSRYm-aG1uVKQGj910BT?usp=sharing

2024/2025 academic year

Statement of use of AI tools

Use of Artificial Intelligence Tools in Assignment Development

We designed the code so that the computer player behaves according to the selected algorithm. However, to support and enhance our development process, we used the following AI tools:

1. ChatGPT (GPT-4o mini, 2025)

- **Link:** https://chatgpt.com/
- **Purpose of Use:**
    - Generated the base code for the Minimax algorithm (without Alpha-Beta Pruning).
    - Provided explanations for how to track the number of developed nodes in the Minimax algorithm.
    - Assisted in debugging the generated code and fixing logical errors.
    - Helped design an experimental timing comparison between Minimax and Alpha-Beta Pruning algorithms via a custom timing.py script.
    - Assisted in understanding heuristic evaluation functions in our group task.

2. ChatGPT (GPT-4, 2023 – Standard Model)

- **Link:** https://chatgpt.com/
- **Purpose of Use:**
    - Offered additional explanations and examples of both Minimax and Alpha-Beta Pruning algorithms.
    - Provided help in debugging intermediate versions of our code.
    - Clarified ideas for implementing a heuristic evaluation function.

3. DeepSeek (Standard Model, 2025)

- **Link:** https://deepseek.com/
- **Purpose of Use:**
    - Suggested suitable data structures for efficient implementation.
    - Helped generate separated versions of the Minimax and Alpha-Beta Pruning algorithms.
    - Explained how to track the total number of nodes developed during the search.

4. Merlin (Standard Model, 2025)

- **Link:** https://merlin.foyer.work/
- **Purpose of Use:**
    - Provided insights into Alpha and Beta cut tracking.

- Assisted in evaluating and comparing the time complexity of Alpha-Beta Pruning vs Minimax.
- Helped debug logical parts of the algorithm and optimize efficiency.

All AI-generated content was thoroughly reviewed, modified, and adapted to fit the specific logic and structure of our game project. The final implementation reflects our own understanding, design, and development process.

Demonstration example of the software

Additional software requirements

The person running the code has to use a python compiler (like Visual Studio Code for example with the python module installed) and have the Kivy package installed to be able to run the UI of the game.

Game description

At the beginning of the game, the player chooses the algorithm the AI will use, either Minimax, which checks all possible moves, or Alpha-Beta pruning, which speeds up the process by skipping unnecessary moves. The player then decides who will start first, either themselves or the AI. Next, the first player enters a starting number between 8 and 18. The game begins with this number, and players take turns multiplying it by 2, 3, or 4. After each move, if the result is an even number, the player loses 1 point; if it is odd, they gain 1 point. If the number ends in 0 or 5, 1 point is added to a bank score. The game continues until the number reaches 1200 or more, at which point the player who made the last move claims all the points in the bank. The game ends by comparing the scores of both players to determine the winner or if the result is a draw. The software also shows statistics such as how many nodes the AI explored, and any alpha or beta cuts made during the process.

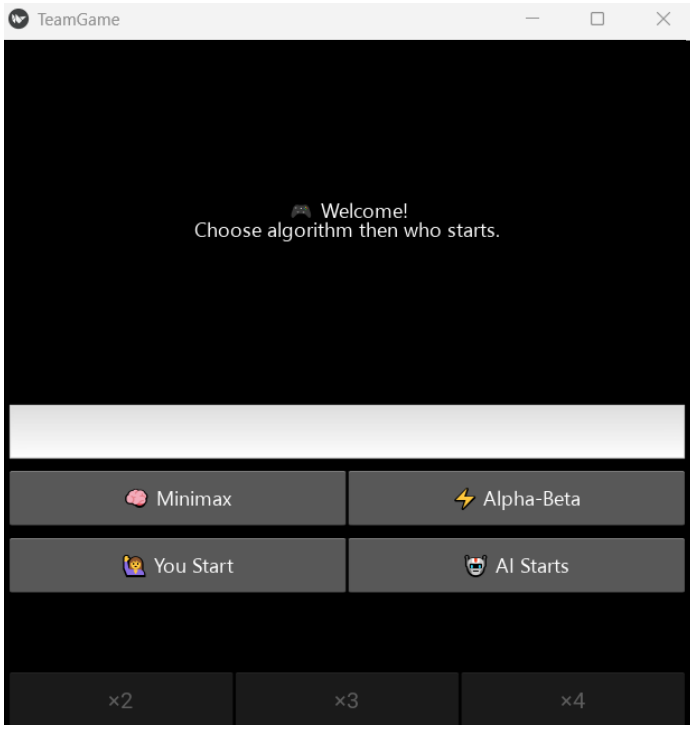
At the end of the game, the human player can play again by clicking on the retry button. Actually, the player can restart the game as soon as the first number was picked. This way, if the player fumbled a move or clicked on a wrong option by accident, he can restart as quick as possible.

By default, the human player is called P1, and the AI is called P2. As demanded in the instructions of this practical assignment, this two-person game has perfect information (all the information about the state of the game is available to both player and they can both predict the end of the game) and opposes an intelligent computer and a human being.

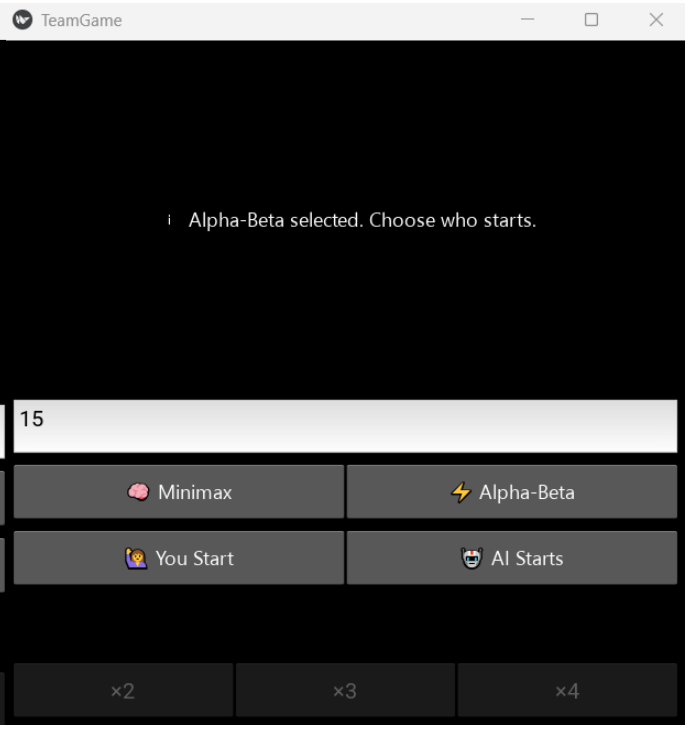
By implementing the minimax and alpha-beta pruning searching algorithms, the computer tracks each move made by both players in order to see changes in the course of the game and make the best move (or at least the one that maximizes its score). We are not using any other search algorithms such as N-ply look ahead.

We are using the Kivy library for Python which will generate the GUI interface for the our game. This way, our UI includes many visual elements such as menu bars, buttons, text fields, icons (emojis) and so on.

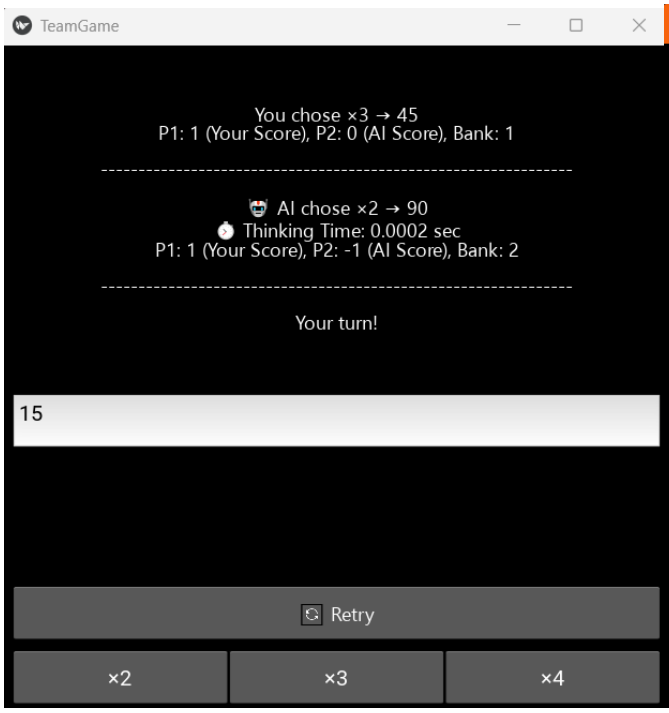
Game screenshots and explanation:



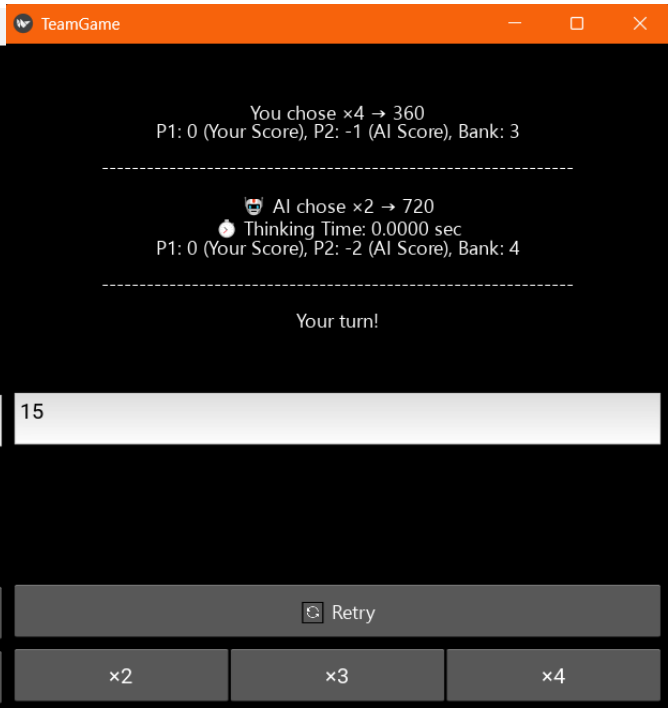
1. Main menu



2. The player inputs a value, select an algorithm and then chooses who starts.

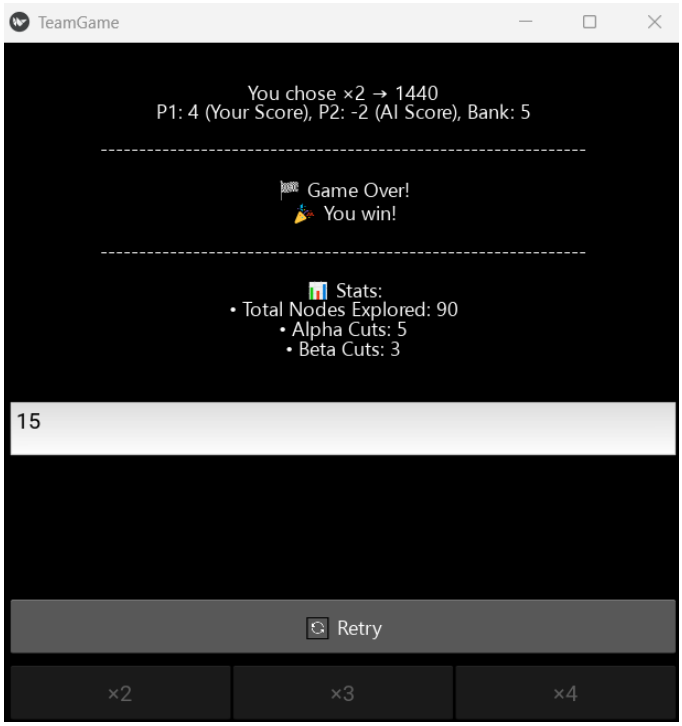


3. Human starts first, he chose 15 as a starting Number and decided to multiply it by 3. AI plays as soon as the human players's last input.



4. Human multiplies the value by 4, AI by 2. Next move is by the human and it will surpass 1200, thus ending the game

5. By choosing 2, the human reaches a value of 1440, higher than 1200, so the game ends. The score of the bank is transfered to his score. The human player wins this game.



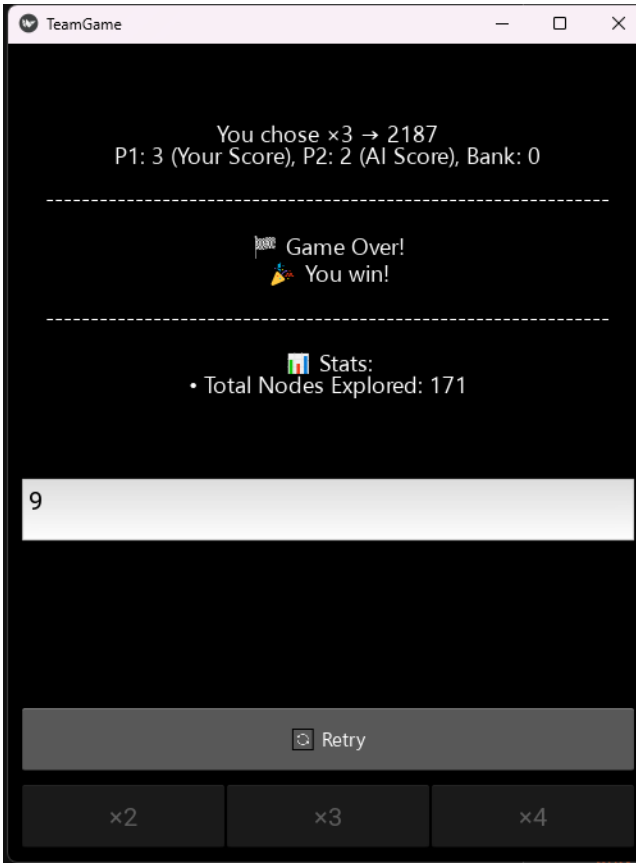
We can see the stats of this game, such as the number of nodes explored by the AI, and the number of shortcuts (alpha-beta pruning) it took.

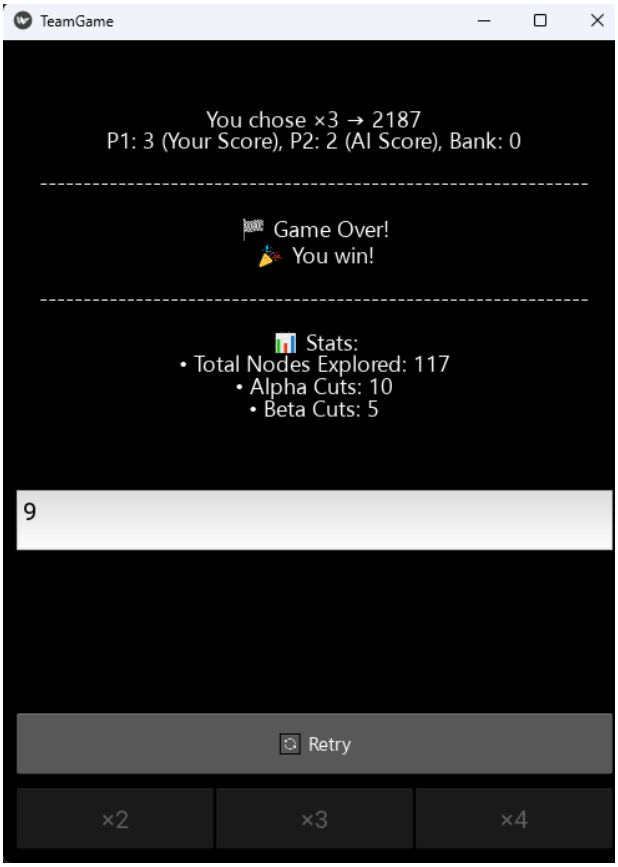
Experiment 1 + 2:
Human Starts 3x-3x-3x (9)

To begin, we selected a random number (9) and conducted experiments with the human player taking the first turn. Two separate runs were performed—one using the **Minimax algorithm** and the other using **Alpha-Beta Pruning**.

- In the **Minimax experiment**, the human player won. The algorithm explored **171 nodes**, and the final score was **2187**.
- In the **Alpha-Beta Pruning experiment**, the human player also won. However, the algorithm only needed to explore **117 nodes**, significantly fewer than Minimax. But the last score (2187) is still the same, the alpha-beta pruning algorithm makes cuts by skipping branches that won't affect the final outcome.

These results clearly demonstrate the efficiency of Alpha-Beta Pruning in reducing the search space while preserving the optimal outcome.



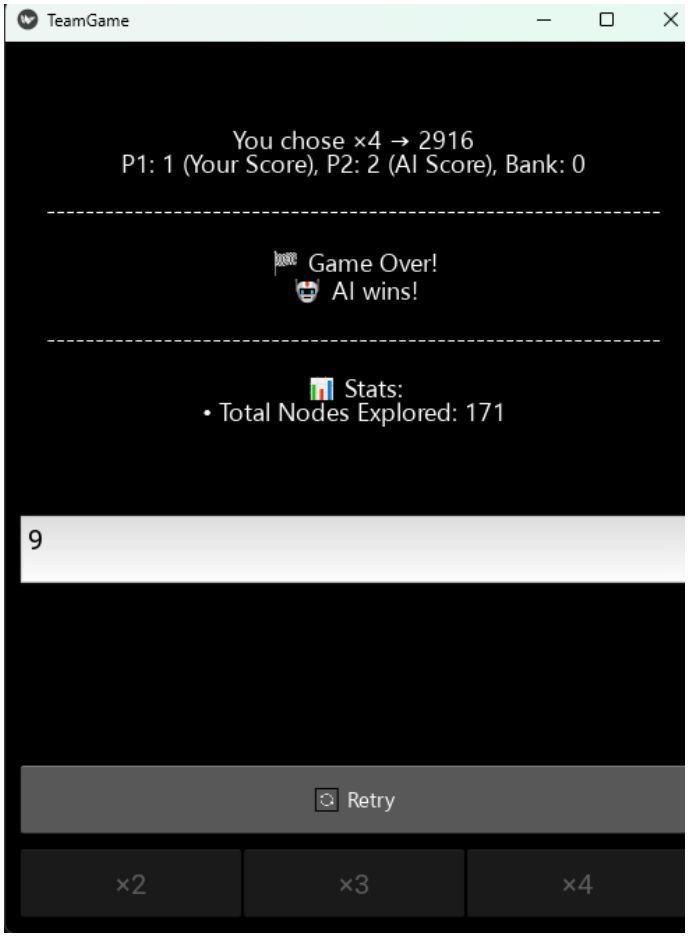


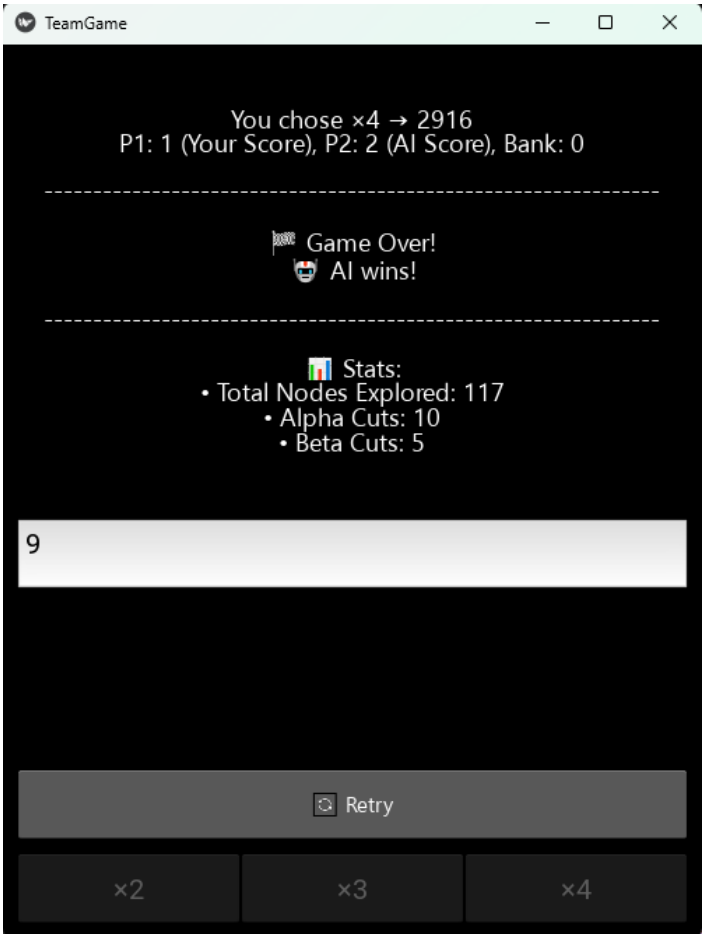
Experiment 3 + 4:

In this round, we also selected the random number **9**, with the human player going first. The human began with the sequence **3x → 3x → 4x**. Since the final move was **4x**, multiplying an odd result by an even number ultimately led to an even number—causing the human to lose the opportunity to secure a winning path. As a result, **the AI wins in both algorithms**.

Moreover, the efficiency of **Alpha-Beta Pruning** is again demonstrated through the number of nodes explored—**only 117 compared to 171** in the Minimax algorithm.

Interestingly, despite choosing the same sequence (**3x → 3x → 4x**) as in Experiment 1, the final scores remained identical. This further confirms that Alpha-Beta Pruning achieves the same outcome with significantly fewer computations, making it the more efficient algorithm.



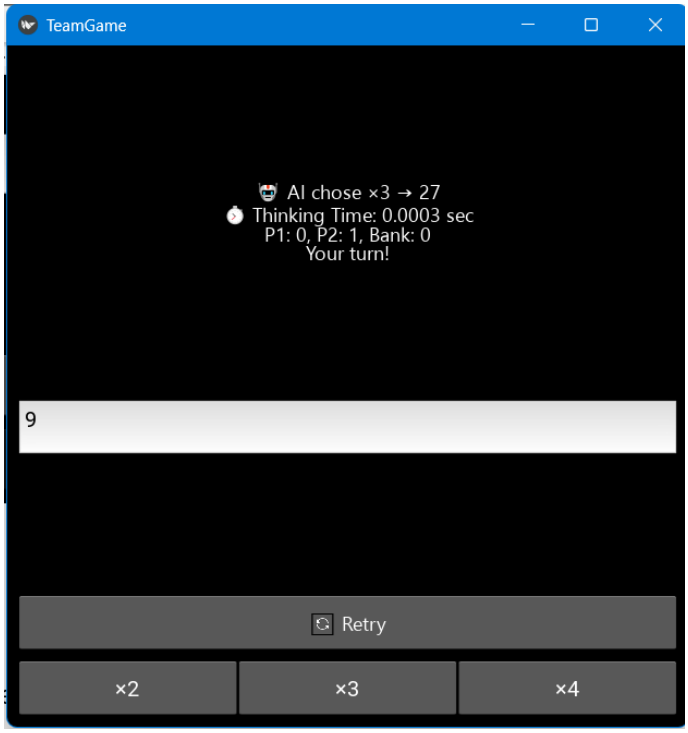


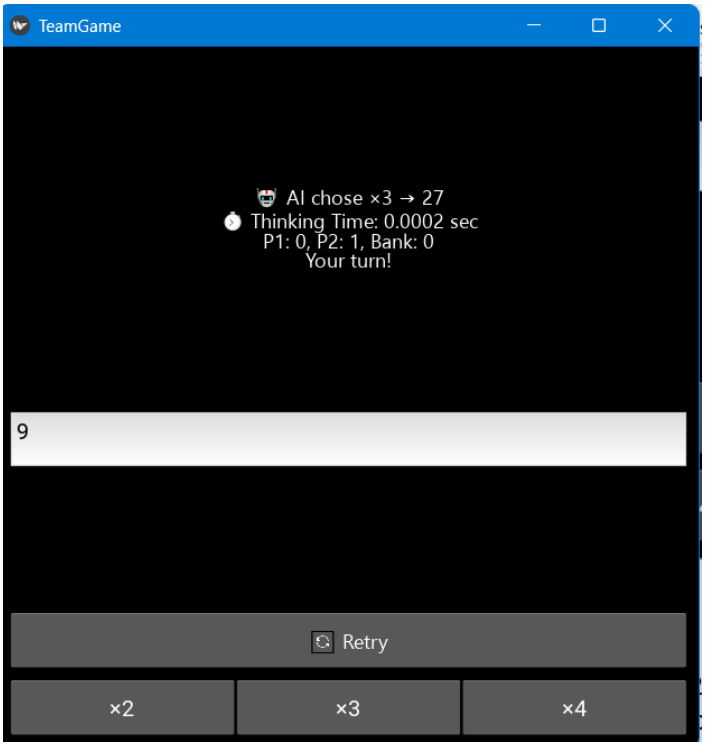
Experiment 5 + 6:

In this scenario, the chosen number is still 9, but this time, the AI takes the first turn.

When comparing both algorithms under identical conditions - the same random number, the same multipliers, and the same move sequence—we observed that Minimax consistently takes more time to process each step than Alpha-Beta Pruning.

For instance, during the first move, Minimax required 0.0003 seconds, while Alpha-Beta Pruning completed the same step in just 0.0002 seconds. This highlights not only the reduced number of nodes explored but also the superior computational efficiency of Alpha-Beta Pruning.





Experiment 7 + 8:

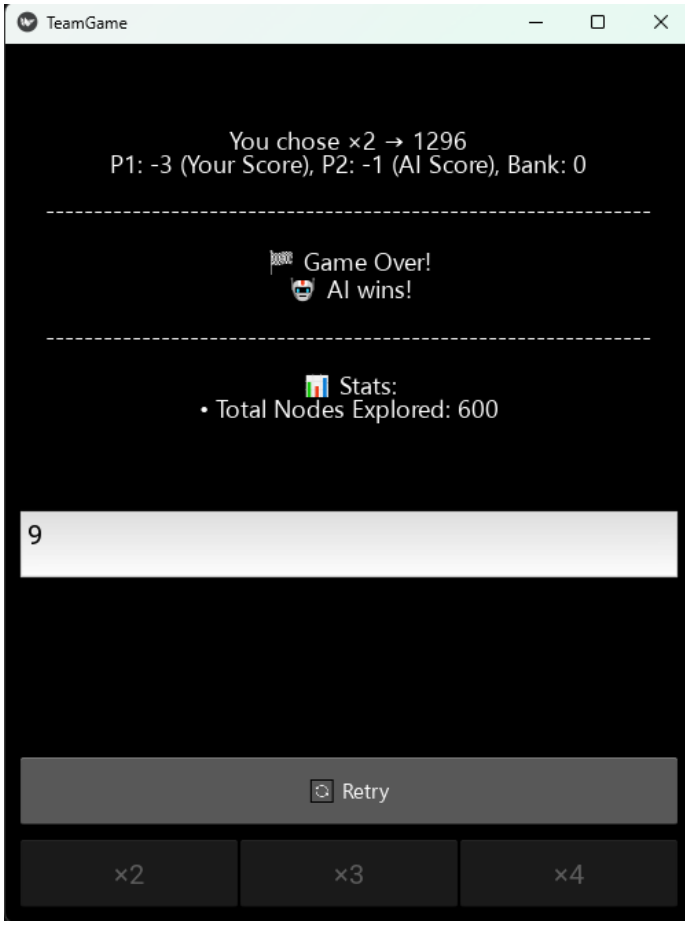
In this experiment, the AI goes first and consistently applies the 2x – 2x – 2x multiplier, starting from the initial number 9.

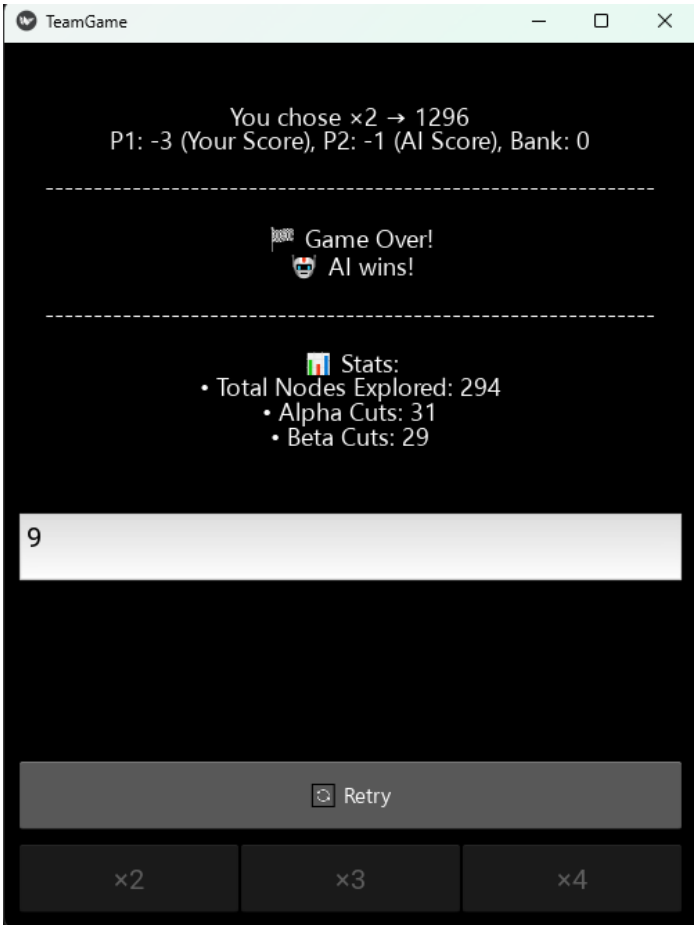
The Minimax algorithm evaluates every possible move in the game tree, resulting in 600 nodes explored, as seen in the first screenshot. In contrast, the Alpha-Beta Pruning algorithm intelligently skips branches that do not influence the final outcome. Thanks to its pruning strategy, it only explores 294 nodes, achieving the same result with significantly less computation. This is evident in the second screenshot, where 31 alpha cuts and 29 beta cuts were made.

In this scenario, the bank score remains at 0 throughout the game. This is because the current multiplier pattern—starting with 9 × 2 - never produces a number ending in 0 or 5, which is the condition required to deposit into the bank. Specifically, 2 multiplying an odd number that is not 5 (like 9) by 2 will always result in an even number (which never ends with 0 or 5) that does not meet the bank condition.

Moreover, once the result becomes even, any subsequent multiplication - whether by an odd or even number- will continue to yield an even number. As a result, the bank value is never triggered, and only the individual scores matter.

In such cases, the player with the less negative score (closer to zero) is considered the winner. This explains why, despite the lack of bank deposits, the game still reaches a conclusion based on the lowest penalty score.





Experiment 9 + 10:

In this experiment, the game begins with the human player and a starting number of 10. The player then applies the 4× multiplier three times consecutively.

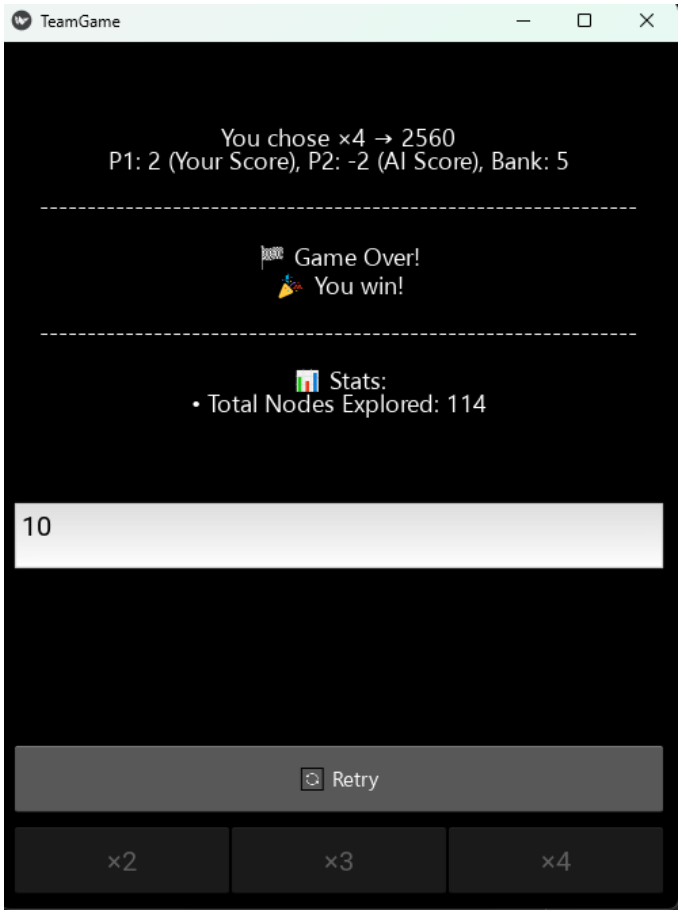
As always, the AI uses either the Minimax algorithm, which explores all possible moves in the game tree, or the Alpha-Beta Pruning algorithm, which skips branches that do not impact the final decision.

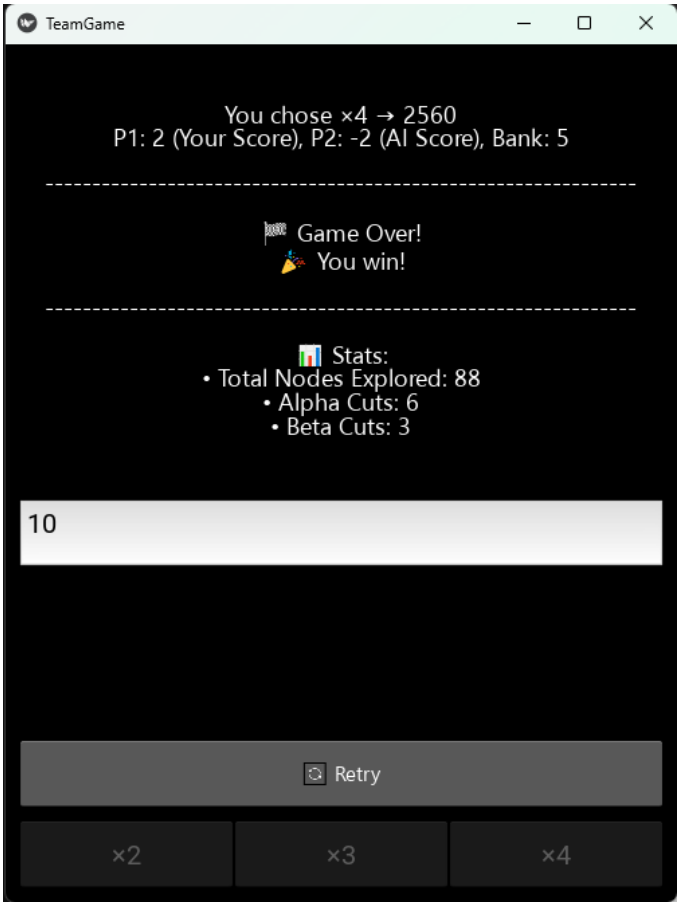
Even though 10 is an even number—which would normally lead both players to accumulate negative scores (since each even result deducts 1 point)—there is an important exception based on the game's rules:

Whenever a move results in a number ending in 0 or 5, 1 point is added to the bank score.

Because 10 multiplied by 2, 3, or 4 repeatedly continues to end in 0 (e.g., $10 \times 4 = 40$, $40 \times 4 = 160$, etc.), the bank score increases consistently throughout the game. Ultimately, when the game ends, the player who made the last move receives all the accumulated bank points.

As a result, even though the human started with an even number and made all even-number-generating moves (which would typically lead to a negative score), the final score becomes positive due to claiming the bank.





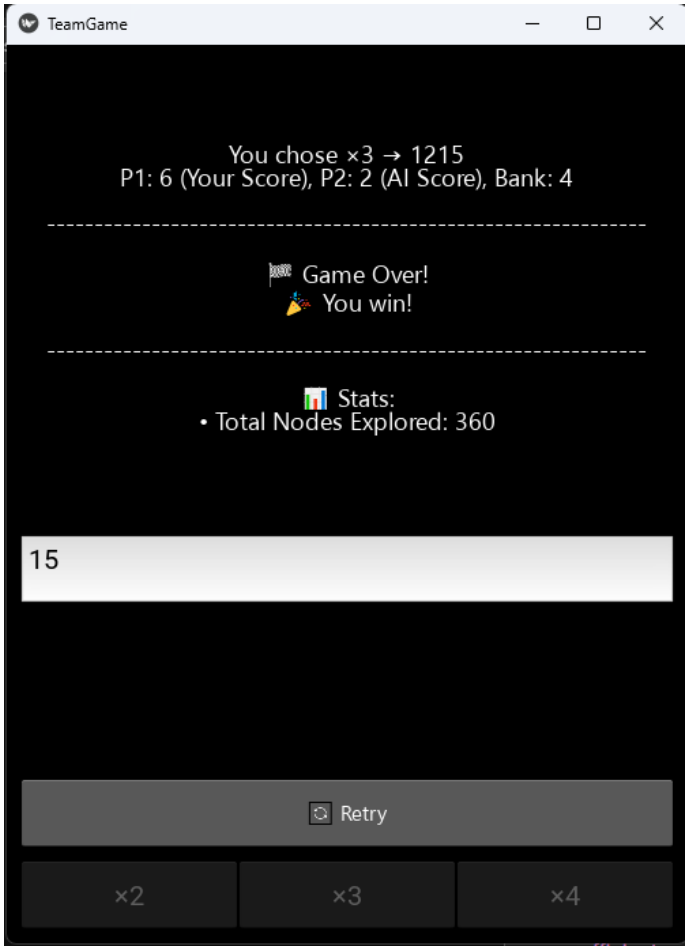
Experiment 11 + 12: WITH AI STARTS

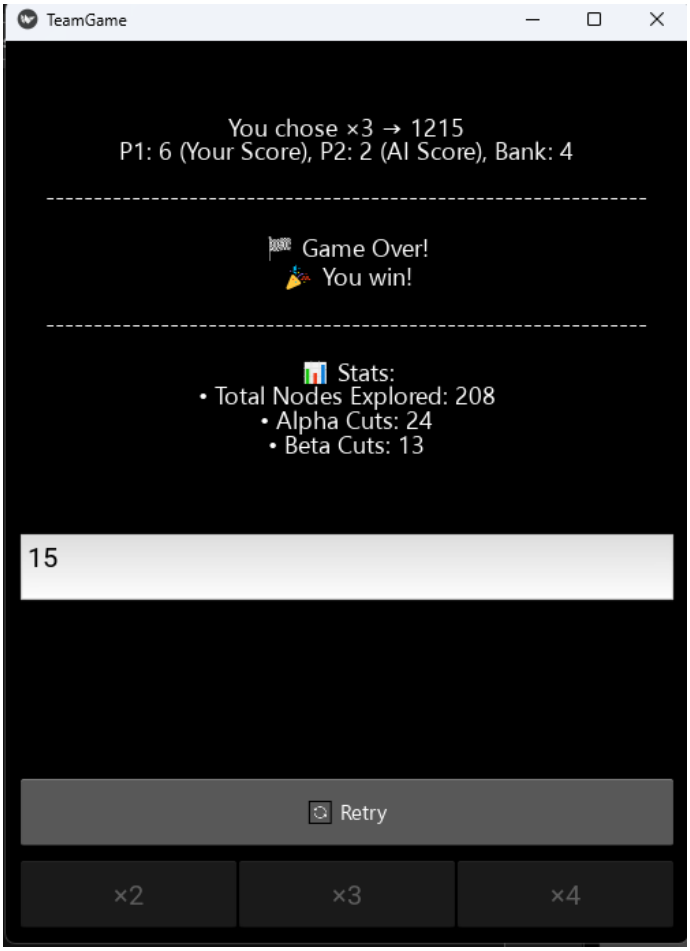
The game starts with the number 15, a larger starting value compared to previous tests. This case is particularly interesting, as 15 behaves similarly to 10 in the context of how the bank score increases.
Here's why:

- When 15 is multiplied by 2 or 4, the result always ends in 0 (e.g., 15 × 2 = 30; 30 × 2 = 60), and any number ending in 0 will continue to end in 0 when multiplied by 2, 3, or 4.
- When 15 is multiplied by 3, the result ends in 5 (e.g., 15 × 3 = 45; 45 × 3 = 135).
- If a number ends in 5, multiplying it again by 2 or 4 will yield a number ending in 0, and multiplying it by 3 will maintain the 5-ending pattern.

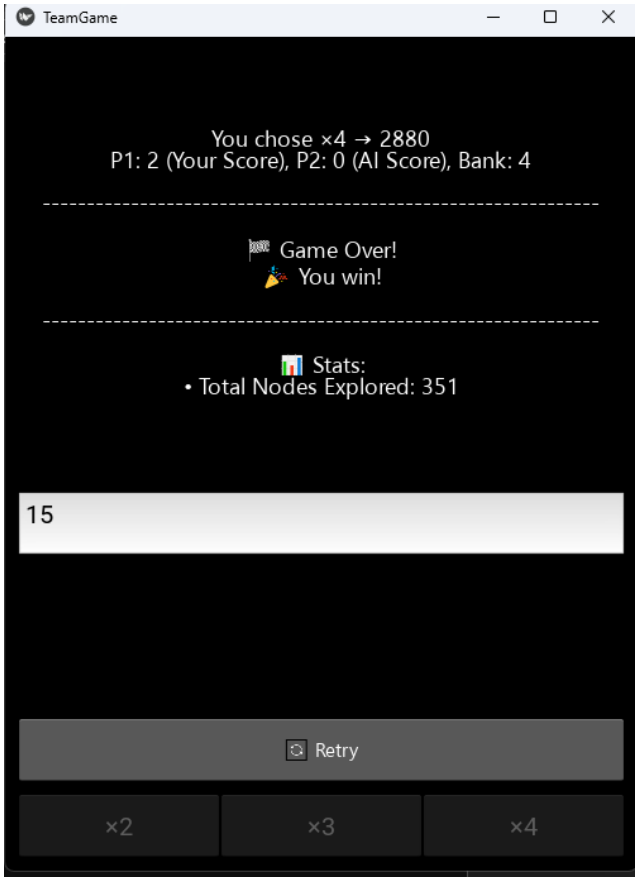
This creates a loop of results ending in either 0 or 5, both of which trigger the bank score condition. Since every move results in a number ending in 0 or 5, the bank score continues to increase steadily throughout the game.
This behavior is clearly reflected in the GUI, allowing us to visually verify that the algorithm handles special cases like 15 correctly, and that the bank accumulation logic functions as expected.

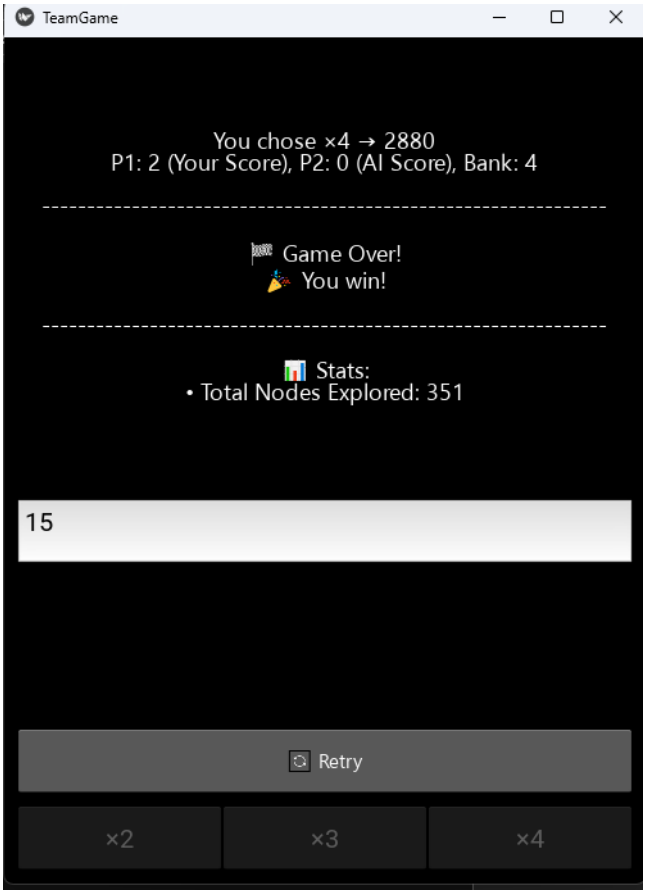
Case 3x 3x 3x:





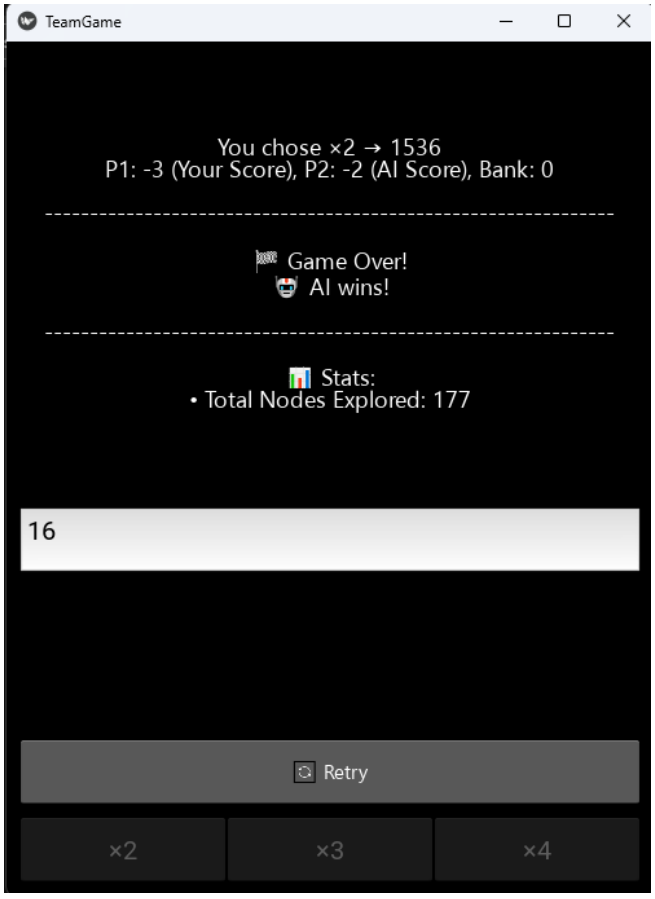
The AI starts with number 15 using we choose 4x-4x:

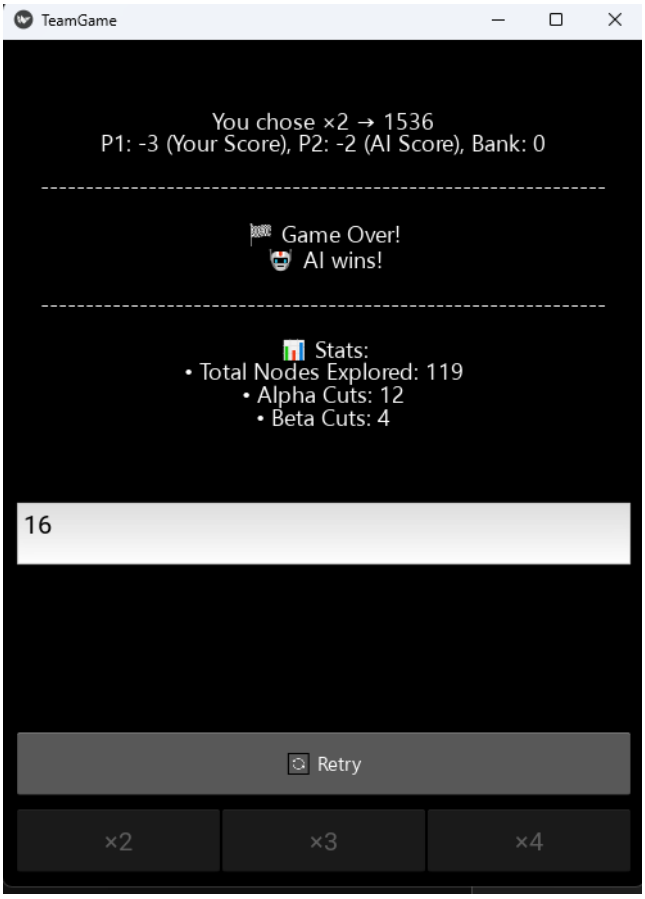




Experiment 13 + 14:

With an initial number of **16** and the human player starting with the sequence **2×** → **2×** → **2×**, our final experiment involved testing the system using a larger value once again to further **validate its accuracy and correctness**.





Conclusions from the Experiments:

Across all of our experiments, the Minimax algorithm consistently took longer to compute and examined a greater number of nodes compared to the Alpha-Beta Pruning algorithm. Even in cases where both approaches led to the same final game outcome, Minimax always explored more branches—this aligns with its theoretical design, as it evaluates all possible paths in the game tree. In contrast, Alpha-Beta Pruning effectively eliminates branches that cannot influence the final decision, significantly reducing the number of nodes explored.
These findings demonstrate that Alpha-Beta Pruning is notably more efficient, particularly as the size and complexity of the game tree increases. As a result, it proves to be a more suitable choice when optimizing for speed and computational resources.

Description of data structures and algorithms

Description of data structures:

We are using Kivy to create the GUI. We are using a custom font for emoji display and also set the display size to (500,420).

We have added the *class TeamGameApp* which will help to run the full game and also it includes the Minimax and Alpha-beta pruning.
Added class which contains all game logics and AI algorithms.

Functions:

Def __init__(self): Help to initialize the game state variable as a default once we started the game.

Def apply_rules(self, currect_game_value, player, score_p1, score_p2, bank): Applies the rules of the game to the current state of the game (adds or subtracts a point to a player, gives a point to the bank if needed).

Def evaluate(self, current_game_value, score_p1, score_p2, bank, is_ia_turn): Evaluates the current game score: if it's over 1200, the game ends and the final score is returned, otherwise it only returns the current score

Def minimax_no_ab(self, current_game_value, score_p1, score_p2, bank, depth, maximizing): Search algorithm that goes through the game tree using the minimax heuristic evaluation. In the "if" condition, we declare the initialize evaluation to "-inf" to the max_eval, because it guarantees any valid move will be better than the initial value. The first evaluated move will always repace "-inf". The subsequent moves only update max_eval if they yield a higher score. It updates the max_eval if the current move is better. A similar algorithm is made for the min_eval, but this time the initialize evaluation number is set to "inf".

Def minimax_alpha_beta(self, current_game_value, score_p1, score_p2, bank, depth, alpha, beta, maximizing): Search algorithm that goes through the game tree using the minimax heuristic evaluation but implements also the alpha-beta pruning algorithm. The algorithm evaluates the heuristic value of the current node, comparing it with the value of alpha (if we are at a maximizing level) or beta (if we are at a minimizing level). In case of maximizing: if we have a current value of alpha higher than the value of beta (of the direct ancestor), then the search algorithm decides to not consider the descendants of this node.
Same applies for the minimizing levels of the game tree.

Def get_valid_moves(self): Returns the set of valid number multipliers (x2, x3, x4).

Def ai_move(self): Determines AI's best move using selected algorithm, measures the time the AI takes to think, and also chooses the best move possible that will maximises its score.
This method determines the AI's best move by evaluting all possible moves which includes Minimax Algorithm or Alpha-beta Algorithm, depending on the users selection. The "False" value in the search algorithms calls indicates that the next move would be played by the AI's opponent, which is Player 1.

Def play_turn(self, multiplier): Executes the game turn with the given multiplier and updates the scores. Also checks for the game end condition and switches the players if game continues.

Def build(self): Builds the application with UI and greeting label, and the input for initial value, algorithm choice buttons, player start choice buttons, and the multiplier options that are disabled initially. It also creates a retry button when the game starts.

Event Handlers:

def choose_minimax(self, instance):
def choose_alpha_beta(self, instance):
def choose_human_start(self, instance):

def choose_ai_start(self, instance):
def start_game(self):
def ai_starts_playing(self):
def on_multiplier_chosen(self, instance):
def get_winner_message(self):
def disable_buttons(self):
def reset_game(self, instance):

These are the functions which can handle all user interactions.
Algorithm selections, starting player selection, game initialization, AI move execution, human move processing, checking if the game value has reached 1200, show the winner of the game, restart the game and disable buttons that are not necessary anymore.

if __name__ == "__main__":
TeamGameApp().run()

This permits us to run the main execution of the Standard Kivy application entry point. And so, the game according to the code.

Description of a heuristic evaluation function: Minimax and Alpha-beta pruning

At first, we decided to draw the game trees of our game to see any pattern that could help us with the development of our heuristic function. Because of the complexity of our game trees, we created the following formula, while taking into account the identified differences that can occur during the game.

With the case, **Case 1: Game Over:**
current_game_value >= 1200:

$$h(s) = \begin{cases} score_{P2} + bank - score_{P1}, if\ AI\ ends\ the\ game, \\ score_{P2} - (score_{P1} + bank), if\ human\ ends\ the\ game \end{cases}$$

With the case, Case 2:
current_game_value < 1200:

$$h(s) = score_{P2} - score_{P1}$$

The full heuristic calculation will be:

$$h(s) = \begin{cases} score_{P2} + bank - score_{P1}, \text{ if AI ends the game, and current\_game\_value >= 1200,} \\ score_{P2} - (score_{P1} + bank), \text{if human ends the game, and current game value >= 1200,} \\ score_{P2} - score_{P1}, \ \ if\ current\_game\_value\ < 1200 \end{cases}$$

Heuristic calculation code:

```
def evaluate(self, current_game_value, score_p1, score_p2, bank, is_ia_turn):
    if current_game_value >= 1200:
        return score_p2 + bank - score_p1 if is_ia_turn else score_p2 - (score_p1 + bank)

    return score_p2 - score_p1
```

Function def evaluate(self, current_game_value, score_p1, score_p2, bank, is_ia_turn): This is what our heuristic evaluation function looks like in our code.

The heuristic evaluates game states differently based on whether the game has reached a terminal condition (current_game_value ≥ 1200) or is still in progress.

For terminal states, if it's the AI's turn when the game ends, the evaluation is calculated as score_p2 + bank - score_p1, crediting the AI with the bank points it would receive.

If it's the player's turn instead, the formula becomes score_p2 - (score_p1 + bank), assuming the player claims the bank points.

For non-terminal states, the heuristic uses a simple score difference: score_p2 - score_p1.

We are using both the minimax and alpha-beta pruning heuristic evaluation functions because of their efficiency to walk through a game tree. It was also imposed in the subject of this practical assignment to have one of them, and to compare their efficiency.
We decided to let the player choose between both heuristic evaluation functions; this way the player can experiment with the time the AI will take to think about its next move.

Description of algorithms:

Full game code available in the appendix 1.

In this game two players (a human and an AI) take turns multiplying the starting number by a factor of 2, 3, or 4. The goal is to reach or exceed 1200 while maximizing the player's score. The AI uses the **Minimax algorithm with Alpha-Beta pruning** to determine the possible move.

AI Move Selection Using Minimax Algorithm

We have to know how AI moves first:

```
def ai_move(self):
    best_score = float('-inf')
    best_move = None
    move_time = 0

    for move in self.get_valid_moves():
        new_val = self.current_game_value * move
        p1, p2, b = self.apply_rules(new_val, 2, self.score_P1, self.score_P2, self.bank)

        start = time.perf_counter()

        if self.use_alpha_beta:
            eval_score = self.minimax(new_val, p1, p2, b, 5, float('-inf'), float('inf'), False)
        else:
            eval_score = self.minimax_no_ab(new_val, p1, p2, b, 5, False)

        end = time.perf_counter()
        duration = end - start

        if eval_score > best_score:
            best_score = eval_score
            best_move = move
            move_time = duration
```

```
    return best_move, move_time
```

In the AI's decision-making process, for every possible move (multiplying the current value by 2, 3, or 4), the game first applies its rules to update the game state, including player scores and the bank. Once this new state is generated, the AI evaluates the move using either the Minimax algorithm or the Alpha-Beta Pruning algorithm, depending on the player's earlier selection. Throughout this process, the AI keeps track of which move leads to the best outcome based on the evaluation score. The search depth is fixed at 5 levels, meaning the AI looks five moves ahead before making a decision. When the evaluation reaches the end of this depth, a scoring function (evaluate()) is used to estimate how good that state is. If Alpha-Beta Pruning is used, the algorithm intelligently skips unnecessary branches, reducing the number of computations without changing the final result. This decision-making process is triggered in two main parts of the game flow: first, when the AI starts the game, handled by the ai_starts_playing() function; and second, when the player makes a move, prompting the AI to respond using the on_multiplier_chosen() function.

The AI selects the best move by exploring a **game tree** using **Minimax with Alpha-Beta pruning**. The algorithm evaluates different move sequences up to a depth of **5 turns ahead**.

Minimax Algorithm with Alpha_Beta Pruning:

1. Maximizing player (AI): Chooses the move that maximizes its score.
2. Minimizing player (Human): Chooses the move that minimizes the AI's score not letting it win the game.
3. Alpha-beta pruning is used to cut off unnecessary branches and speed up computation.

Mini-Max:

Mini-max algorithm is one of the simplest search-based algorithms that is used in turn-based games. In this game the players are considered Maximizer and Minimizer where the Maximizer aims to maximize the score while the Minimizer aims to minimize it. The algorithm recursively explores all possible game states and assigns values to them, choosing the optimal move based on these evaluations. (Lecture notes, Module 3.3)

Implementation of Min-max:

```
def minimax_no_ab(self, current_game_value, score_p1, score_p2, bank, depth, maximizing):
        if self.current_player == 2:
            self.total_nodes_explored += 1

        if current_game_value >= 1200 or depth == 0:
            return self.evaluate(current_game_value, score_p1, score_p2, bank, maximizing)

        if maximizing:
            max_eval = float('-inf')
            for move in self.get_valid_moves():
                new_val = current_game_value * move
                p1, p2, b = self.apply_rules(new_val, 2, score_p1, score_p2, bank)
                eval = self.minimax_no_ab(new_val, p1, p2, b, depth - 1, False)
                max_eval = max(max_eval, eval)
            return max_eval
        else:
            min_eval = float('inf')
            for move in self.get_valid_moves():
                new_val = current_game_value * move
```

```
            p1, p2, b = self.apply_rules(new_val, 1, score_p1, score_p2, bank)
            eval = self.minimax_no_ab(new_val, p1, p2, b, depth - 1, True)
            min_eval = min(min_eval, eval)
        return min_eval
```

In the above code it is shown how minimax (without alpha-beta) is being used.
The recursive function minimax_no_ab:

- simulates different **possible moves** by multiplying the current game value by **2, 3, or 4**.
- **alternates between the AI (maximizing) and the player (minimizing)**.
- If the game reaches **1200** or the maximum search depth, the function returns a **heuristic evaluation**.

In this situation each move results in a new current_game_value.

If the new value is **greater than 1200**, the turn is skipped. Moreover,

if current_game_value ends in **0**, the banked score is **collected and**

if the new value is **prime**, the player gets **bonus points**.

In this code AI as a Maximizer selects the move with the highest score while Human player chooses the lowest score as Minimizer. The function **recursively explores all possible future game states** and chooses the best path.

Alpha-beta:

Alpha-beta pruning is an alternative to the minimax algorithm that improves the search efficiency of the state space graph of the game. It is based on the idea that moves that are not beneficial (or in other words, moves that make no difference to the final decision) can be ignored (Jones, 2009; Russell and Norvig, 2010). Alpha-beta pruning therefore returns the same move as minimax but cuts off unpromising paths from the state space graph (Luger, 2009). As a result, it does not give heuristic measures for all nodes.

This algorithm uses the same rules for propagating heuristic measures (Luger, 2009):

- if a node is located at MAX level, it acquires the maximum heuristic measure from among its direct descendants;
- if a node is located at MIN level, it acquires the minimum heuristic measure from among its direct descendants.

Two numbers are used in alpha-beta pruning (Jones, 2009; Luger, 2009):

- **An alpha number (α)** is assigned to nodes at MAX levels. This value should not decrease, since this would represent a bad move. Hence, this number defines the best move that can be made to maximise the result of the game;
- **A beta number (β)** is assigned to nodes at MIN levels. It should not increase, since this would represent a bad move. This number therefore defines the best move that can be done to minimise the result of the game.

The algorithm uses two rules to perform cut-offs on the basis of the α and β values (Luger, 2009):

- Unconsidered paths of the state space graph are cut off below a node at a MIN level, if the β value for this node is less than or equal to the value of α (i.e. $\beta \leq \alpha$) for its direct ancestors (referred to as an **α-cut-off**);

- Unconsidered paths of the state space graph are cut off below a node at a MAX level, if the value of α for this node is greater than or equal to the value of β (i.e. α≥β) for its direct ancestors (referred to as a **β-cut-off**). (Lecture Notes, Module)

Implementation of Alpha-beta pruning:

```
def minimax(self, current_game_value, score_p1, score_p2, bank, depth, alpha, beta, maximizing):

    if self.current_player == 2:
        self.total_nodes_explored += 1


    if current_game_value >= 1200 or depth == 0:
        return self.evaluate(current_game_value, score_p1, score_p2, bank, maximizing)

    if maximizing:
        max_eval = float('-inf')
        for move in self.get_valid_moves():
            new_val = current_game_value * move
            p1, p2, b = self.apply_rules(new_val, 2, score_p1, score_p2, bank)
            eval = self.minimax(new_val, p1, p2, b, depth - 1, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)

            if beta <= alpha:
                self.alpha_cuts += 1  # Always count alpha cuts
                break
        return max_eval
    else:
        min_eval = float('inf')
        for move in self.get_valid_moves():
            new_val = current_game_value * move
            p1, p2, b = self.apply_rules(new_val, 1, score_p1, score_p2, bank)
            eval = self.minimax(new_val, p1, p2, b, depth - 1, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)

            if beta <= alpha:
                self.beta_cuts += 1
                break
        return min_eval
```

When it comes to Alpha-beta pruning, it is being used to optimize the previous algorithm by cutting off unnecessary moves.

1. **Recursive Function (minimax)**:
    a. Similar to Minimax but introduces two additional parameters:
        - alpha → Best score the maximizer (AI) can guarantee.
        - beta → Best score the minimizer (player) can guarantee.
2. **Alpha-Beta Pruning in Action**:
    - As the algorithm explores different moves, it **compares the scores with alpha and beta**.
    - If a branch **cannot improve the AI's choice**, it is **pruned (skipped)**.

- This significantly **reduces the number of moves that need to be evaluated**.

3. **Tracking Pruning**:
   - **Alpha cuts**: If the maximizer finds a value **greater than or equal to beta**, further exploration is stopped.
   - **Beta cuts**: If the minimizer finds a value **less than or equal to alpha**, further exploration is stopped.

In this game the AI Calls Minimax or Alpha-Beta: When it's the AI's turn, it checks all possible moves (multiplying the number by 2, 3, or 4). It simulates the future outcomes and assigns scores.

Evaluating the Best Move: The AI chooses the move that leads to the highest evaluation score. If Alpha-Beta Pruning is enabled, unnecessary paths are skipped for faster decision-making.

Returning the Best Move: Once all moves are evaluated, the AI selects the best option and executes it.

Comparison of algorithms:

Minimax and alpha pruning what happen if we just only use Minimax, only Alpha pruning, write the code version

The minimax algorithm is one of the simplest of the search-based game algorithms. It involves the following steps (Luger, 2009):

1. create a complete state space graph;
2. label the levels of the graph as MIN and MAX;
3. assign heuristic measures to all leaf nodes according to an analysis of who wins at a specific node;
4. propagate heuristic measures of leaf nodes up to the root node of the state space graph according to the following rules: a) if a node is located at MAX level, it acquires the maximum heuristic measure from among its direct descendants, and b) if a node is located at MIN level, it acquires the minimum heuristic measure from among its direct descendants;
5. determine the result of the game and the winning paths.

Alpha-beta pruning is an alternative to the minimax algorithm that improves the search efficiency of the state space graph of the game. Lecture notes of Module 3
It is based on the idea that moves that are not beneficial (or in other words, moves that make no difference to the final decision) can be ignored (Jones, 2009; Russell and Norvig, 2010).
Alpha-beta pruning therefore returns the same move as minimax, but cuts off unpromising paths from the state space graph (Luger, 2009). As a result, it does not give heuristic measures for all nodes.

Alpha-beta pruning is clearly more efficient, and we found that it is 1,75x quicker than the minimax algorithm.

Information sources

Appendix 1

All software code that corresponds to the implementation of the game and not to the creation of the graphical interface

```
import time


class TeamGameAppConsole:
    def __init__(self):
        self.score_P1 = 0
        self.score_P2 = 0
        self.bank = 0
        self.current_game_value = None
        self.current_player = 1
        self.use_alpha_beta = True  # Use Alpha-Beta Pruning or Minimax


        # Performance stats
        self.total_nodes_explored = 0
        self.alpha_cuts = 0
        self.beta_cuts = 0


    def apply_rules(self, current_game_value, player, score_p1, score_p2, bank):
        if current_game_value % 2 == 0:
            if player == 1:  # Player 1
                score_p1 -= 1
            else:  # Player 2 or AI
                score_p2 -= 1
        else:
            if player == 1:
                score_p1 += 1
            else:
                score_p2 += 1
```

```
    if current_game_value % 10 in {0, 5}:
        bank += 1

    return score_p1, score_p2, bank

def evaluate(self, current_game_value, score_p1, score_p2, bank, is_ia_turn):
    if current_game_value >= 1200:  # Endgame condition
        return score_p2 + bank - score_p1 if is_ia_turn else score_p2 - (score_p1 + bank)
    return score_p2 - score_p1

def minimax_no_ab(self, current_game_value, score_p1, score_p2, bank, depth, maximizing):
    self.total_nodes_explored += 1

    if current_game_value >= 1200 or depth == 0:
        return self.evaluate(current_game_value, score_p1, score_p2, bank, maximizing)

    if maximizing:  # AI's turn
        max_eval = float('-inf')
        for move in self.get_valid_moves():
            new_val = current_game_value * move
            p1, p2, b = self.apply_rules(new_val, 2, score_p1, score_p2, bank)
            eval = self.minimax_no_ab(new_val, p1, p2, b, depth - 1, False)
            max_eval = max(max_eval, eval)
        return max_eval
    else:  # Player's turn
        min_eval = float('inf')
        for move in self.get_valid_moves():
            new_val = current_game_value * move
            p1, p2, b = self.apply_rules(new_val, 1, score_p1, score_p2, bank)
```

```
            eval = self.minimax_no_ab(new_val, p1, p2, b, depth - 1, True)
            min_eval = min(min_eval, eval)
        return min_eval

    def minimax(self, current_game_value, score_p1, score_p2, bank, depth, alpha, beta, maximizing):
        self.total_nodes_explored += 1

        if current_game_value >= 1200 or depth == 0:
            return self.evaluate(current_game_value, score_p1, score_p2, bank, maximizing)

        if maximizing:  # AI's turn
            max_eval = float('-inf')
            for move in self.get_valid_moves():
                new_val = current_game_value * move
                p1, p2, b = self.apply_rules(new_val, 2, score_p1, score_p2, bank)
                eval = self.minimax(new_val, p1, p2, b, depth - 1, alpha, beta, False)
                max_eval = max(max_eval, eval)
                alpha = max(alpha, eval)
                if beta <= alpha:  # Pruning
                    self.alpha_cuts += 1
                    break
            return max_eval
        else:  # Player's turn
            min_eval = float('inf')
            for move in self.get_valid_moves():
                new_val = current_game_value * move
                p1, p2, b = self.apply_rules(new_val, 1, score_p1, score_p2, bank)
                eval = self.minimax(new_val, p1, p2, b, depth - 1, alpha, beta, True)
                min_eval = min(min_eval, eval)
                beta = min(beta, eval)
```

```
            if beta <= alpha:  # Pruning
                self.beta_cuts += 1
                break
        return min_eval

def get_valid_moves(self):
    return [2, 3, 4]

def ai_move(self):
    start_time = time.time()  # Start time measurement
    best_score = float('-inf')
    best_move = None

    for move in self.get_valid_moves():
        new_val = self.current_game_value * move
        p1, p2, b = self.apply_rules(new_val, 2, self.score_P1, self.score_P2, self.bank)

        if self.use_alpha_beta:
            eval_score = self.minimax(new_val, p1, p2, b, 5, float('-inf'), float('inf'), False)
        else:
            eval_score = self.minimax_no_ab(new_val, p1, p2, b, 5, False)

        if eval_score > best_score:
            best_score = eval_score
            best_move = move

    end_time = time.time()  # End time measurement
    time_taken = end_time - start_time
    print(f"🤖 AI took {time_taken:.4f} seconds to choose the multiplier.")
```

```
        return best_move

    def play_turn(self, multiplier):
        self.current_game_value *= multiplier
        self.score_P1, self.score_P2, self.bank = self.apply_rules(
            self.current_game_value, self.current_player, self.score_P1, self.score_P2, self.bank
        )

        if self.current_game_value >= 1200:
            if self.current_player == 1:
                self.score_P1 += self.bank
            else:
                self.score_P2 += self.bank
            return True
        self.current_player = 2 if self.current_player == 1 else 1
        return False

    def start_game(self):
        print("🎮 Welcome to the Team Game!\n")

        # Select starting number
        while True:
            try:
                value = int(input("Enter a starting number (8-18): "))
                if 8 <= value <= 18:
                    self.current_game_value = value
                    break
                else:
                    print("❌ Enter a number between 8 and 18.")
            except ValueError:
```

```
            print("❌ Invalid input. Please enter a valid number.")


        # Choose algorithm
        while True:
            algo_choice = input("Choose Algorithm:\n1: Minimax\n2: Alpha-Beta Pruning\nEnter choice (1 or 2): ").strip()
            if algo_choice in {"1", "2"}:
                self.use_alpha_beta = algo_choice == "2"
                break
            else:
                print("❌ Invalid input. Please enter 1 or 2.")


        # Choose starting player
        while True:
            player_choice = input("Who should start? (1: You, 2: AI): ").strip()
            if player_choice in {"1", "2"}:
                self.current_player = int(player_choice)
                break
            else:
                print("❌ Invalid input. Please choose 1 or 2.")


        # Game loop
        while self.current_game_value < 1200:
            print(f"\nCurrent state: Value = {self.current_game_value}, P1 = {self.score_P1}, P2 = {self.score_P2}, Bank = {self.bank}")


            if self.current_player == 1:  # Human's turn
                while True:
                    try:
                        multiplier = int(input("Your turn! Choose a multiplier (2, 3, or 4): "))
```

```
                if multiplier in self.get_valid_moves():
                    break
                else:
                    print("❌ Invalid multiplier. Choose 2, 3, or 4.")
            except ValueError:
                print("❌ Invalid input. Enter a number (2, 3, or 4).")
        game_over = self.play_turn(multiplier)

    else:  # AI's turn
        print("🤖 AI is thinking...")
        multiplier = self.ai_move()
        print(f"🤖 AI chose ×{multiplier}")
        game_over = self.play_turn(multiplier)

    if game_over:
        break

# Announce the result
self.announce_result()

def announce_result(self):
    if self.score_P1 > self.score_P2:
        print("\n🎉 You win!")
    elif self.score_P2 > self.score_P1:
        print("\n🤖 AI wins!")
    else:
        print("\n🤝 It's a tie!")

    print(f"\n📊 Stats:\n- Total Nodes Explored: {self.total_nodes_explored}")
```

```
        if self.use_alpha_beta:
            print(f"- Alpha Cuts: {self.alpha_cuts}\n- Beta Cuts: {self.beta_cuts}")


if __name__ == "__main__":
    game = TeamGameAppConsole()
    game.start_game()
```

Appendix 2

<in case of using artificial intelligence tools in the development of the assignment, add screenshots of all the prompts entered by the student team in a specific AI tool and all the answers provided by the tool; both the prompts and the answers must be visible in full and in good quality>

When we debug (English):
https://chatgpt.com/share/67ebe975-fb70-8010-81e8-aa65b5666739

https://chatgpt.com/share/67ec47f6-8b8c-8012-88a9-b577bd024c47

First creation of the code French:
https://chatgpt.com/share/67ecce83-4610-800d-8df1-2fc5229bf1a5

Translation and modifications of the code (French):
https://chatgpt.com/share/67eccebc-b6cc-800d-9bce-1fad0ed2ac27

Explaining the code for us (Vietnamese):
https://chatgpt.com/share/67eecbfa-5a98-8012-8d19-0edc4f0593fd

Ask explaining alpha-beta pruning in the code (Vietnamese):
https://chatgpt.com/share/67eecc44-6e94-8012-b3b1-24cb2d33953a

Debug (Vietnamese):
https://chatgpt.com/share/67eecca9-1bc8-8012-a96f-bd76778f97bb

https://chatgpt.com/share/67eeccc9-caf4-8012-b09a-924c688db0ba

https://chatgpt.com/share/67eecceb-84b0-8012-a3bf-8db6b61c48ca

https://chatgpt.com/share/67eecd17-be80-8012-8c14-4a978dc79cfa

https://chatgpt.com/c/67e2bffd-45cc-8012-80bd-a8dc07658e99